

Callwave: Complete Standards Implementation Map

IEEE Software Standards + IEEE Networking Standards + IETF RFCs + ITU-T Standards

PART 1: IEEE SOFTWARE DEVELOPMENT STANDARDS

Overview: Software Development & Quality Assurance

IEEE software standards define **how to develop, test, and maintain** the Callwave application. They cover the entire software lifecycle from requirements gathering to deployment and maintenance.

1. IEEE 830 - Software Requirements Specification (SRS)

Purpose: Define WHAT the system must do

Scope for Callwave:

- Functional requirements (login, calls, recordings, etc.)
- Non-functional requirements (performance, security, scalability)
- User stories with acceptance criteria
- System constraints (Cameroon-specific network limits)
- Regulatory/compliance requirements

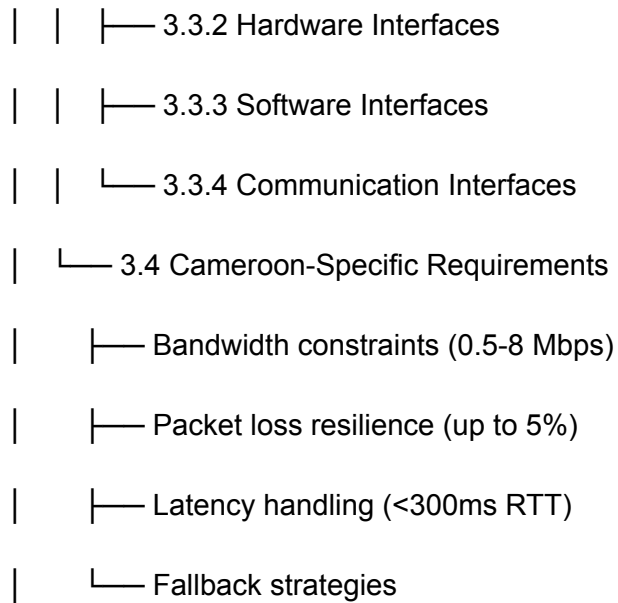
Implementation Locations:

File: docs/IEEE_830_SRS.md

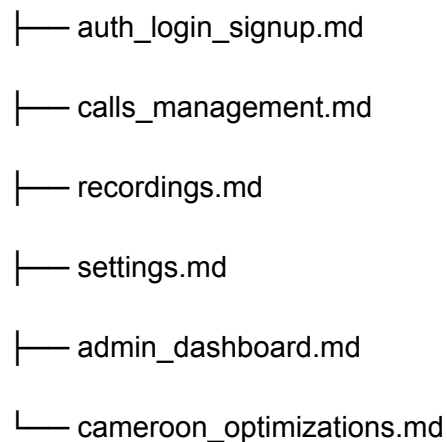
| — 1. Introduction

| | — Purpose

- | | — Scope
- | | — Definitions, acronyms, abbreviations
- | | — References
- | — 2. Overall Description
- | | — Product perspective
- | | — Product functions
- | | — User characteristics
- | | — Constraints
- | — 3. Specific Requirements
- | | — 3.1 Functional Requirements
- | | | — 3.1.1 Authentication (Login/Signup)
- | | | — 3.1.2 Call Management
- | | | — 3.1.3 Recording & Playback
- | | | — 3.1.4 Admin Dashboard
- | | — 3.2 Non-Functional Requirements
- | | | — 3.2.1 Performance (RFC 8083, ITU-T Y.1541)
- | | | — 3.2.2 Security (RFC 3711 SRTP)
- | | | — 3.2.3 Scalability (1000 concurrent)
- | | | — 3.2.4 Reliability (99.5% uptime)
- | | | — 3.2.5 Usability
- | | — 3.3 Interface Requirements
- | | | — 3.3.1 User Interface



File: sprints/user-stories/



Who Uses This:

- Product managers (define scope)
- Developers (know what to build)
- QA (write test cases)
- Business stakeholders (verify completeness)

Output: Clear, testable requirements before coding starts

2. IEEE 1016 - Software Design Specification (SDS)

Purpose: Define HOW the system is built

Scope for Callwave:

- Architecture (Flutter + Node.js + Mediasoup SFU)
- Module design (services, repositories, providers)
- Data flow diagrams
- API contracts (REST + WebSocket)
- Design decisions & rationale
- Technology choices with justification

Implementation Locations:

File: docs/IEEE_1016_SDS.md

├─ 1. Design Overview

| └─ Architecture diagram (3-tier: Flutter, Node.js, Mediasoup)

| └─ Component interactions

| └─ Technology stack rationale

├─ 2. Detailed System Design

| └─ 2.1 Client Layer (Flutter)

| | └─ Module structure (lib/features/*, lib/core/*)

| | └─ State management (Riverpod)

| | └─ Navigation (go_router)

| | └─ Data models

| └─ 2.2 API Layer (Node.js Express)

| | └─ REST endpoints

| | └─ WebSocket events

- | | | — Request/response schemas
- | | | — Error handling
- | | — 2.3 Media Layer (Mediasoup SFU)
- | | | — Producer/Consumer model
- | | | — Transport layer
- | | | — RTP/RTCP handling
- | | | — Bitrate adaptation
- | | — 2.4 Database Layer (PostgreSQL)
- | | | — Schema design
- | | | — Relationships
- | | | — Indexing strategy
- | | | — Migration procedures
- | — 3. Interface Specifications
- | | — 3.1 API Endpoints (RFC 4566, 3550)
- | | — 3.2 WebSocket Messages
- | | — 3.3 Data Transfer Objects (DTOs)
- | | — 3.4 Error Responses
- | — 4. Design Rationale
- | | — Why Mediasoup? (vs Janus, Kurento)
- | | — Why Node.js? (native Mediasoup integration)
- | | — Why Flutter? (cross-platform desktop-first)

| |— Why PostgreSQL? (ACID, scalability)

| |— RFC/ITU-T compliance justification

|— 5. Non-Functional Design

|— 5.1 Security (RFC 3711 SRTP, TLS)

|— 5.2 Performance (RFC 8083 congestion control)

|— 5.3 Scalability (horizontal scaling)

|— 5.4 Reliability (failover, recovery)

File: architecture/

|— flutter-architecture.md

|— nodejs-architecture.md

|— database-schema.sql

|— api-specification.md

|— deployment-architecture.md

Who Uses This:

- Backend architects (design services)
- Frontend architects (design components)
- DevOps (design infrastructure)
- Developers (follow design patterns)

Output: Clear blueprint for developers to follow

3. IEEE 1012 - Software Verification & Validation (V&V)

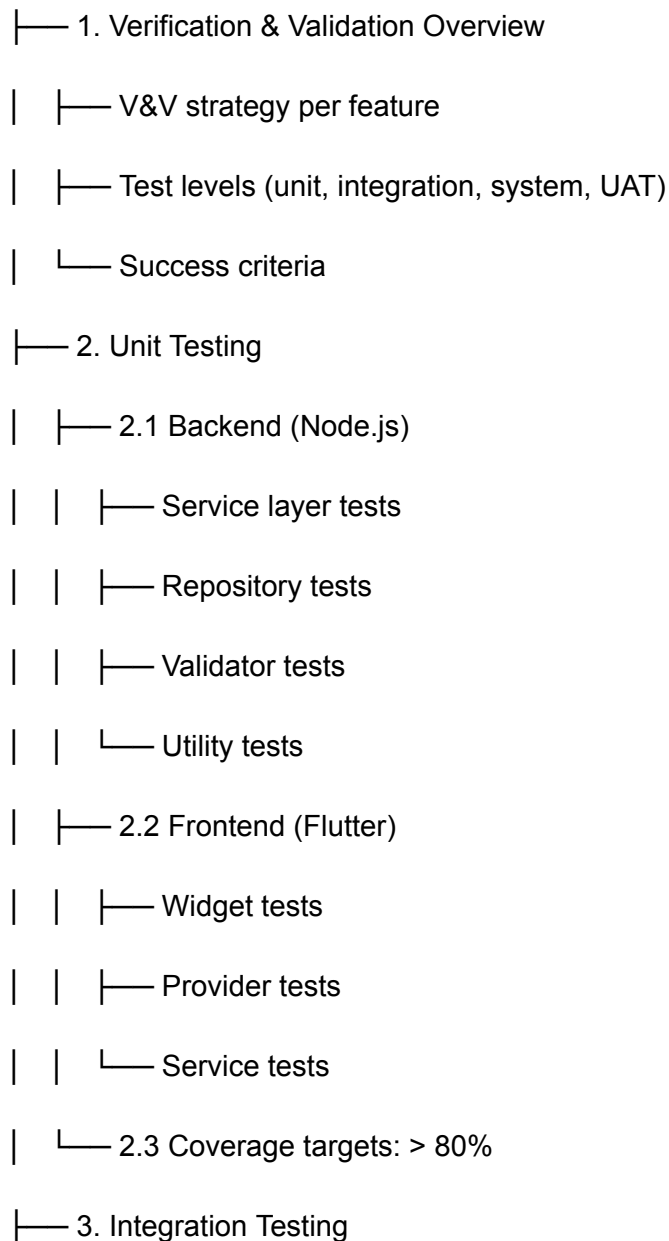
Purpose: Define HOW we verify the system works correctly

Scope for Callwave:

- Test strategy (unit, integration, end-to-end)
- Test case design
- Acceptance criteria verification
- Quality metrics
- Defect management
- Test coverage targets

Implementation Locations:

File: docs/IEEE_1012_VV_Plan.md



- | | — 3.1 API + Database
- | | — 3.2 WebSocket + Mediasoup
- | | — 3.3 Authentication flow
- | | — 3.4 Call flow (end-to-end)
- | — 4. System Testing
 - | | — 4.1 Performance testing (RFC 2889)
 - | | — 4.2 Stress testing (1000 concurrent)
 - | | — 4.3 Network simulation (RFC 8083)
 - | | — 4.4 Security testing (RFC 3711 SRTP)
- | — 5. User Acceptance Testing (UAT)
 - | | — 5.1 Test scenarios per user story
 - | | — 5.2 Cameroon network conditions
 - | | — 5.3 Acceptance criteria
- | — 6. Quality Metrics
 - | | — Code coverage (target: > 80%)
 - | | — Defect density (< 3 per 1000 lines)
 - | | — Test pass rate (target: 100% for release)
 - | | — MOS score (target: > 4.0, ITU-T P.800)
- | — 7. Defect Management
 - | | — Classification (critical, major, minor)
 - | | — Severity assessment
 - | | — Fix & re-test procedure

└─ Root cause analysis

File: test/

└─ unit/

| └─ services/

| └─ repositories/

| └─ utils/

└─ integration/

| └─ auth_flow_test.dart

| └─ call_flow_test.dart

| └─ api_integration_test.js

└─ system/

| └─ performance_test.js

| └─ stress_test.js

| └─ network_simulation_test.js

| └─ security_test.js

└─ uau/

└─ scenarios/

└─ reports/

Who Uses This:

- QA engineers (write test cases)
- Developers (run tests before commit)
- Test automation engineers (CI/CD)
- Release managers (verify quality gates)

Output: Comprehensive test plan with >80% coverage

4. IEEE 829 - Software Test Documentation

Purpose: Document all testing activities and results

Scope for Callwave:

- Test plan (scope, schedule, resources)
- Test case specifications
- Test procedures
- Test results & reports
- Test summary & metrics

Implementation Locations:

File: docs/IEEE_829_Test_Documentation.md

|— 1. Test Plan

| |— 1.1 Scope (what to test)

| |— 1.2 Approach (how to test)

| |— 1.3 Resources (tools, personnel)

| |— 1.4 Schedule (test timeline)

| |— 1.5 Deliverables

|— 2. Test Case Specifications

| |— Format: Test Case ID, description, preconditions, steps, expected result

| |— 2.1 Authentication tests (20 test cases)

| |— 2.2 Call management tests (50 test cases)

| |— 2.3 Recording tests (15 test cases)

- | | — 2.4 Admin dashboard tests (25 test cases)

- | | — 2.5 Performance tests (10 test cases)

- | | — 2.6 Security tests (15 test cases)

- | — 3. Test Procedures

- | | — Test execution steps

- | | — Environment setup

- | | — Data preparation

- | | — Cleanup procedures

- | — 4. Test Results & Reports

- | | — 4.1 Daily test summary

- | | — 4.2 Defect reports (with screenshots)

- | | — 4.3 Test execution log

- | | — 4.4 Coverage metrics

- | | — 4.5 Test completion report

- | — 5. Test Metrics & Analysis

- | | — Pass/fail rate

- | | — Defect density

- | | — Test coverage (code, feature)

- | | — MOS scores (ITU-T P.800)

- | | — Network metrics (RFC 2889)

- | | — Recommendations

File: test-reports/

```
|— daily/
| |— 2024-09-01-test-summary.md
| |— 2024-09-02-test-summary.md
| |— ...
|— test-cases/
| |— TC-AUTH-001-successful-login.md
| |— TC-CALL-001-create-call.md
| |— TC-PERF-001-1000-concurrent.md
| |— ...
|— defect-reports/
|   |— DEF-001-login-validation.md
|   |— DEF-002-audio-dropout.md
|   |— ...
```

Who Uses This:

- QA leads (manage test execution)
- Test engineers (execute tests, log results)
- Project managers (track quality metrics)
- Stakeholders (verify readiness for release)

Output: Complete test documentation with traceability to requirements

5. IEEE 1028 - Software Reviews & Audits

Purpose: Ensure quality through systematic reviews and audits

Scope for Callwave:

- Code reviews (peer review before merge)
- Design reviews (architecture validation)
- Test reviews (test plan completeness)
- Compliance audits (RFC/IEEE/ITU-T adherence)
- Process audits (following standards)

Implementation Locations:

File: docs/IEEE_1028_Reviews_Audits.md

| — 1. Review Procedures

| | — 1.1 Code Review Process

| | | — Trigger: Before merging to main

| | | — Reviewers: 2+ developers

| | | — Checklist: style, logic, security

| | | — Approval: Lead developer

| | | — Documentation: Review notes

| | — 1.2 Design Review

| | | — Trigger: Major architecture changes

| | | — Participants: Architects, leads

| | | — Deliverables: Design doc, decisions

| | | — Approval: CTO

| | — 1.3 Test Review

| | | — Trigger: Before UAT

| | | — Review scope: Test coverage, cases

| | | — Approval: QA Lead

- | | └─ Compliance: >80% coverage target

- | └─ 1.4 Compliance Review

- | | └─ RFC/IEEE/ITU-T adherence

- | | └─ Security requirements (RFC 3711)

- | | └─ Performance targets (ITU-T Y.1541)

- | └─ Documentation completeness

- └─ 2. Audit Checklists

- | └─ 2.1 Code Quality Audit

- | | └─ Code style consistency

- | | └─ No hardcoded secrets

- | | └─ Error handling completeness

- | | └─ Null pointer checks

- | | └─ Resource cleanup

- | | └─ Comments/documentation

- | └─ 2.2 Security Audit

- | | └─ SRTP enabled (RFC 3711)

- | | └─ TLS for all transport

- | | └─ Input validation

- | | └─ SQL injection prevention

- | | └─ Authentication/authorization

- | | └─ Audit logging

- | └─ 2.3 Performance Audit

- | | | — Bitrate adaptation (RFC 8083)
- | | | — Latency < 300ms (ITU-T Y.1541)
- | | | — Jitter < 30ms
- | | | — Packet loss recovery (FEC)
- | | — CPU/memory usage
- | — 2.4 Compliance Audit
 - | — IEEE 830 SRS completeness
 - | — IEEE 1016 SDS adherence
 - | — Test coverage (IEEE 1012)
 - | — Documentation (IEEE 829)
 - | — RFC/ITU-T implementation
- 3. Review Records
 - | — 3.1 Review meeting notes
 - | — 3.2 Reviewer checklists (completed)
 - | — 3.3 Findings & corrective actions
 - | — 3.4 Approval sign-offs
 - | — 3.5 Follow-up actions
- 4. Audit Frequency
 - Code reviews: Every PR (continuous)
 - Design reviews: Monthly or on major changes
 - Test reviews: Pre-UAT, pre-release

- └─ Compliance audits: Monthly

- └─ Process audits: Quarterly

File: reviews/

- └─ code-reviews/

- └─ PR-001-authentication.md

- └─ PR-002-call-management.md

- └─ ...

- └─ design-reviews/

- └─ 2024-09-architecture-review.md

- └─ ...

- └─ test-reviews/

- └─ 2024-09-test-coverage-review.md

- └─ ...

- └─ compliance-audits/

- └─ 2024-09-rfc-compliance-audit.md

- └─ 2024-09-ieee-adherence-audit.md

- └─ ...

- └─ audit-checklists/

- └─ code-quality-checklist.md

- └─ security-checklist.md

- └─ performance-checklist.md

- └─ compliance-checklist.md

Who Uses This:

- Technical leads (conduct reviews)
- QA leads (audit testing)
- Project managers (track compliance)
- Process improvement team (optimize workflows)

Output: Documented proof of quality assurance compliance

Summary: IEEE Software Standards Implementation

Standard	Purpose	Deliverable	Owner	Frequency
IEEE 830	Define requirements	SRS document	Product/PM	Once (before coding)
IEEE 1016	Design system	SDS document	Architect	Per major release
IEEE 1012	Verify quality	V&V plan + test results	QA Lead	Continuous
IEEE 829	Document tests	Test docs + reports	QA Engineer	Per test cycle
IEEE 1028	Audit compliance	Review records	Tech Lead	Continuous + Monthly

PART 2: NETWORK QUALITY & MEDIA STANDARDS

Overview: Networking, QoS, Codecs & Quality

Network standards define **how media is transported, encoded, and quality is measured** across networks. These are critical for reliable video/audio calls, especially in Cameroon's bandwidth-constrained environment.

TL;DR: Quick Reference Table

IEEE SOFTWARE STANDARDS (Development & Quality)

Standard	Focus	Primary Document	Key Deliverable	Purpose
IEEE 830	Requirements	docs/IEEE_830_SRS.md	Functional + non-functional requirements	Define WHAT to build
IEEE 1016	Design	docs/IEEE_1016_SDS.md	Architecture, module design, API contracts	Define HOW to build
IEEE 1012	Testing	docs/IEEE_1012_VV_Plan.md	Test strategy, >80% code coverage	Verify it works correctly
IEEE 829	Test Documentation	docs/IEEE_829_Test_Documentation.md	Test cases, results, metrics	Document all testing
IEEE 1028	Reviews & Audits	docs/IEEE_1028_Reviews_Audits.md	Audit checklists, review records	Ensure quality compliance

IEEE NETWORKING STANDARDS (QoS & Layer 2/3)

|---|---|---|---| | IEEE 802.11 | L1-L2 | WiFi Device Mgmt |
[lib/core/services/device_service.dart](#) | WiFi quality detection | | IEEE 802.1p | L2 |
Socket Priority | [server/src/config/socket-config.js](#) | LAN packet prioritization (Class
6 for voice/video) | | IEEE 802.1Q | L2 | VLAN Config | [infrastructure/vlan-setup.sh](#) |
VLAN tagging + 802.1p CoS |

IETF RFCs (Media Transport & Signaling)

Standard	Layer	Component	Primary File	Purpose
RFC 3550	L4	Mediasoup SFU	server/src/services/mediasoup-wrapper.js	RTP packet handling (SSRC, timestamps)
RFC 3551	L4	Codec Payload	server/src/config/codec-config.js	H.264 payload type 96, Opus payload type 111
RFC 3711	L4	DTLS/SRTP	server/src/services/srtp-handler.js	AES-128-CM + HMAC-SHA1 encryption
RFC 3389	L4	Audio Processing	server/src/services/comfort-noise.js	Comfort noise (payload type 13) during silence
RFC 4566	L5	Signaling	server/src/services/sdp-handler.js	SDP offer/answer for session setup
RFC 5389	L5	NAT Traversal	infrastructure/coturn/coturn.conf	STUN server (port 3478)
RFC 5766	L5	NAT Traversal	infrastructure/coturn/coturn.conf	TURN relay server + credentials

Standard	Layer	Component	Primary File	Purpose
RFC 5245	L5	ICE Selection	lib/core/services/ice_service.dart	Priority: host(126) > srflx(100) > relay(0)
RFC 6679	L3	QoS Marking	server/src/config/dscp-config.js	DSCP EF(46) for RTP, AF41(34) for data
RFC 8083	L5	Congestion Control	server/src/services/bandwidth-estimator.js	Transport-wide CC, adaptive bitrate
RFC 2889	Monitoring	Network Testing	server/src/monitoring/network-metrics.js	Throughput, latency, jitter, packet loss

ITU-T STANDARDS (Codecs & Quality Metrics)

Standard	Layer	Component	Primary File	Purpose
ITU-T H.264	L6	Video Codec	server/src/config/codec-h264.js	Baseline/Main/High profile, 90kHz clock
ITU-T Opus	L6	Audio Codec	server/src/config/codec-audio.js	48kHz, 6-128 kbps adaptive, FEC enabled
ITU-T P.800	L7	Analytics	server/src/services/qoe-calculator.js	MOS score (1-5), ITU-T G.107 E-model
ITU-T E.800	L7	Analytics	server/src/services/qoe-tracker.js	QoE metrics per call

Standard	Layer	Component	Primary File	Purpose
ITU-T Y.1541	Monitoring	Performance Targets	server/src/config/performance-targets.js	Latency <300ms, jitter <30ms, loss <1%

ARCHITECTURE LAYERS & STANDARDS

Layer 7: Application (QoE, Analytics)	
ITU-T P.800 (MOS), E.800 (QoE), Y.1541 (Targets)	
Layer 6: Codec (Audio/Video Encoding)	
ITU-T H.264, Opus (IETF RFC 7587), G.711, G.722	
Layer 5: Media Transport & Signaling	
RFC 3550 (RTP), 3551 (Payloads), 3711 (SRTP)	
RFC 4566 (SDP), 5389 (STUN), 5766 (TURN), 5245 (ICE)	
RFC 8083 (Congestion Control)	

Layer 4: Network Layer QoS & Routing	
RFC 6679 (DSCP), RFC 2889 (Benchmarking)	
IEEE 802.1p (LAN Priority), 802.1Q (VLAN)	

Layer 3: Network Infrastructure	
IEEE 802.11 (WiFi), 802.3 (Ethernet)	
Nginx, Linux kernel TC (Traffic Control)	

DETAILED IMPLEMENTATION BY STANDARD

SECTION A: IEEE SOFTWARE STANDARDS IMPLEMENTATION

IEEE 830 - SRS Implementation

- **Location:** [docs/IEEE_830_SRS.md](#)
- **Updated:** When requirements change
- **Audience:** All team members
- **Status:** Living document

IEEE 1016 - SDS Implementation

- **Location:** [docs/IEEE_1016_SDS.md](#)
- **Updated:** Per major release
- **Audience:** Architects, senior developers
- **Status:** Living document

IEEE 1012 - V&V Implementation

- **Location:** [docs/IEEE_1012_VV_Plan.md](#), [test/](#) directories
- **Updated:** Per test cycle
- **Audience:** QA team
- **Status:** Continuous

IEEE 829 - Test Documentation Implementation

- **Location:** [docs/IEEE_829_Test_Documentation.md](#), [test-reports/](#)
- **Updated:** Per test cycle
- **Audience:** QA, developers
- **Status:** Continuous

IEEE 1028 - Reviews & Audits Implementation

- **Location:** [docs/IEEE_1028_Reviews_Audits.md](#), [reviews/](#)
- **Updated:** Continuous (per PR/review)
- **Audience:** Tech leads, QA
- **Status:** Continuous

SECTION B: NETWORK & MEDIA STANDARDS IMPLEMENTATION

1. RFC 3550 - RTP (Real-time Transport Protocol)

Where: Mediasoup SFU (backend) + flutter_webrtc (client)

Implementation:

File: `server/src/services/mediasoup-wrapper.js`

- SSRC generation (unique per producer)

- Sequence number tracking (0-65535)
- Timestamp calculation: 90kHz for video, 48kHz for audio
- RTCP Sender/Receiver Reports parsing

File: lib/core/services/mediasoup_service.dart

- Receives RTP streams via flutter_webrtc
 - Plays out received audio/video
 - Sends producer stream with RTP headers
-

2. RFC 3551 - RTP Payload Formats

Where: Codec configuration

Implementation:

File: server/src/config/codec-config.js

Video:

- H.264: payload type 96, clock rate 90000

Params: profile-level-id=42e01e, packetization-mode=1

- VP8: payload type 98, clock rate 90000

Audio:

- Opus: payload type 111, clock rate 48000

Params: useinbandfec=1 (Forward Error Correction)

- G.711: payload type 0, clock rate 8000
-

3. RFC 3711 - SRTP (Secure RTP)

Where: DTLS key exchange → SRTP encryption

Implementation:

File: server/src/services/srtp-handler.js

- DTLS establishes TLS handshake
- Derives master key + master salt
- AES-128 Counter Mode (AES-CM) encrypts RTP payload
- HMAC-SHA1 authenticates (10-byte tag)

File: Flutter (automatic via WebRTC)

- flutter_webrtc handles SRTP transparently
 - All media encrypted by default
-

4. RFC 3389 - Comfort Noise

Where: Audio silence handling

Implementation:

File: server/src/services/comfort-noise.js

- Detect silence in audio stream
 - Generate CN packet (payload type 13)
 - Level byte (0-127) indicates noise intensity
 - Prevents caller perception of "dead air"
-

5. RFC 4566 - SDP (Session Description Protocol)

Where: Session negotiation between client & Mediasoup

Implementation:

File: server/src/services/sdp-handler.js

- Generate SDP offer with codecs, RTP maps
- Parse remote SDP for capabilities
- Exchange via WebSocket

Example SDP lines:

v=0

m=video 9 UDP/TLS/RTP/SAVP 96 98

a=rtpmap:96 H264/90000

a=fmtp:96 level-asymmetry-allowed=1;profile-level-id=42e01e

6. RFC 5389 - STUN (NAT Detection)

Where: Coturn server + ICE gathering

Implementation:

File: infrastructure/coturn/coturn.conf

listening-port=3478

realm=cameroon-stun.callwave.cm

fingerprint

File: lib/core/services/ice_service.dart

- flutter_webrtc queries STUN server

- Returns XOR-MAPPED-ADDRESS (client's public IP:port)
 - Result used as server reflexive ICE candidate
-

7. RFC 5766 - TURN (NAT Relay)

Where: Coturn relay server for restrictive NAT

Implementation:

File: infrastructure/coturn/coturn.conf

relay-ip=10.0.1.100

external-ip=203.x.x.x/10.0.1.100

user=callwave_relay:password

File: server/src/services/turn-service.js

- Generate time-limited credentials (expires in 1 hour)
- Username: [timestamp]:[userId]
- Password: HMAC-SHA1(secret, username)

File: lib/core/services/call_service.dart

- Fetch TURN credentials from backend
 - Use as fallback when direct path fails
-

8. RFC 5245 - ICE (Connectivity Establishment)

Where: Candidate gathering & selection

Implementation:

File: lib/core/services/ice_service.dart

Priority order (highest first):

1. Host (local IP) - priority 126
2. Server Reflexive (STUN) - priority 100
3. Peer Reflexive (discovered) - priority 98
4. Relay (TURN) - priority 0

Connectivity checks performed automatically

First successful candidate pair selected

9. RFC 6679 - DiffServ (DSCP) QoS Marking

Where: End-to-end packet prioritization

Implementation:

File: server/src/config/dscp-config.js

- EF (Expedited Forwarding): 46 (0x2E) - calls
- AF41 (Assured Forwarding): 34 (0x22) - video conference
- BE (Best Effort): 0 - background

File: infrastructure/tc-qdisc.sh (Linux Traffic Control)

```
tc qdisc add dev eth0 root handle 1: htb default 10
```

```
tc filter add ... handle 2E fw classid 1:30 # EF → high priority queue
```

```
tc filter add ... handle 22 fw classid 1:20 # AF41 → medium
```

```
tc filter add ... handle 00 fw classid 1:10 # BE → low
```

File: infrastructure/nginx-qos.conf

```
location /rtsp {  
  
    so_priority 6; # Set Linux socket priority (maps to 802.1p class 6)  
  
}
```

10. RFC 8083 - Congestion Control

Where: Adaptive bitrate algorithm

Implementation:

File: server/src/services/bandwidth-estimator.js

- Monitor RTCP feedback (RFC 4585 transport-wide CC)
- Estimate available bandwidth from packet loss/delay
- Adjust producer max bitrate dynamically

Cameroon-specific thresholds:

- < 1 Mbps: 800 kbps video
 - 1-2 Mbps: 1.5 Mbps
 - 2-5 Mbps: 3 Mbps
 - > 5 Mbps: 5 Mbps
-

11. IEEE 802.1p - LAN Priority Levels

Where: Socket priority + network switch

Implementation:

File: server/src/config/socket-config.js

```
const dgram = require('dgram');
```

```
const server = dgram.createSocket('udp4');

// Set SO_PRIORITY = 6 (maps to IEEE 802.1p class 6: Voice/Video)

server.setOption(3, 6, 6);

// On network switch:

# vlan 100 - Voice/Video

# priority 6 (highest priority queue)
```

12. IEEE 802.11 - WiFi Quality

Where: Client device monitoring

Implementation:

File: lib/core/services/device_service.dart

- getWiFiSignalStrength() - RSSI monitoring
- isWiFi5GHz() - Detect WiFi generation
- adjustBitrateForWiFiQuality() - Adapt to WiFi conditions

File: lib/core/services/bitrate_monitor.dart

- Monitor RTCStatsReport periodically
 - Detect WiFi degradation
 - Reduce bitrate on poor WiFi
-

13. ITU-T H.264 - Video Codec

Where: Video encoding/decoding

Implementation:

File: server/src/config/codec-h264.js

Profiles:

- Baseline (42e00a): Low power devices
- Main (4d401e): Balanced
- High (640028): Maximum quality

Cameroon optimization:

- Start with Baseline
- Probe to Main on good network
- Multi-resolution: 1280x720 > 854x480 > 426x240

Bitrate: 500kbps (min) - 8Mbps (max)

Framerate: 15-30 fps

14. ITU-T Opus - Audio Codec

Where: Audio encoding/decoding

Implementation:

File: server/src/config/codec-audio.js

- Clock rate: 48kHz
- Channels: 2 (stereo)
- FEC enabled (useinbandfec=1)

Bitrate adaptation (Cameroon):

- 6 kbps: Minimum (emergency)
- 16 kbps: Voice only (poor network)

- 32 kbps: Voice + moderate quality
 - 64 kbps: Wideband voice
 - 128 kbps: Stereo music quality
-

15. ITU-T P.800 - MOS (Mean Opinion Score)

Where: Call quality calculation

Implementation:

File: server/src/services/qoe-calculator.js

$MOS = f(RTT, \text{jitter}, \text{packet loss})$

Formula (ITU-T G.107 E-model, simplified):

$R_o = 93.2$ (ideal)

$R_o -= \text{latencyImpairment}(RTT)$

$R_o -= \text{jitterImpairment}(\text{jitter})$

$R_o -= \text{packetLossImpairment}(\text{loss})$

R-factor → MOS conversion:

$MOS = 1 + 0.035 \cdot R + 0.000007 \cdot R \cdot (R - 60) \cdot (100 - R)$

MOS scale:

- 5: Excellent (> 4.2)
- 4: Good (3.5 - 4.2)
- 3: Fair (2.5 - 3.5)
- 2: Poor (1.5 - 2.5)

- 1: Bad (< 1.5)

Target: > 4.0 (Good)

16. ITU-T Y.1541 - Performance Targets

Where: SLA monitoring

Implementation:

File: server/src/config/performance-targets.js

Targets:

- Latency: < 150ms one-way (< 300ms RTT)
- Jitter: < 20ms (allowance: 30ms)
- Packet loss: < 0.1% (allowance: 1%)
- Availability: 99.5% (Cameroon-realistic)

Cameroon-specific:

- Bandwidth minimum: 500 kbps
 - Fallback to audio if video fails
 - FEC + adaptive bitrate enabled
-

17. RFC 2889 - Benchmarking

Where: Network testing & metrics

Implementation:

File: server/src/monitoring/network-metrics.js

- Throughput: bits/second delivered

- Latency: round-trip time (ms)
- Jitter: delay variance (ms)
- Packet loss: % of packets lost
- Burst loss: consecutive packets lost
- Bandwidth utilization: % link capacity

File: test/network-benchmark.js

- Test scenarios: ideal, Cameroon-typical, poor
- Measure codec recovery effectiveness
- Verify FEC performance
- Benchmark TURN relay overhead

FILE STRUCTURE SUMMARY

SOFTWARE STANDARDS DOCUMENTS

Callwave/

| — docs/

| | — IEEE_830_SRS.md ★ SOFTWARE STANDARD

| | | — Full requirements specification

| | — IEEE_1016_SDS.md ★ SOFTWARE STANDARD

| | | — Architecture & design specification

| | — IEEE_1012_VV_Plan.md ★ SOFTWARE STANDARD

| | | — Test strategy & verification plan

- | | — IEEE_829_Test_Documentation.md ★ SOFTWARE STANDARD
- | | — Test cases, procedures, results
- | — IEEE_1028_Reviews_Audits.md ★ SOFTWARE STANDARD
- | — Review procedures, audit checklists
- | — reviews/ ★ SOFTWARE STANDARD (Implementation)
- | | — code-reviews/
- | | — design-reviews/
- | | — test-reviews/
- | | — compliance-audits/
- | | — audit-checklists/
- | — sprints/ ★ SOFTWARE STANDARD (SRS Implementation)
- | | — user-stories/
- | | | — auth_login_signup.md
- | | | — calls_management.md
- | | | — ...
- | — test/ ★ SOFTWARE STANDARD (1012 Implementation)
- | | — unit/
- | | — integration/
- | | — system/
- | | — uau/

NETWORK & MEDIA STANDARDS IMPLEMENTATION

Callwave/

```
|— lib/
|   |— core/services/
|       |— device_service.dart      🌐 IEEE 802.11 WiFi
|       |— ice_service.dart        🌐 RFC 5245 ICE
|       |— mediasoup_service.dart  🌐 RFC 3550 RTP
|       |— call_service.dart       🌐 RFC 4566 SDP
|       |— bitrate_monitor.dart    🌐 RFC 8083 Congestion
|       |— quality_indicator.dart  🌐 ITU-T P.800 MOS display
|
|— server/src/
|   |— config/
|       |— codec-config.js         🌐 RFC 3551 Payloads
|       |— codec-h264.js           🌐 ITU-T H.264
|       |— codec-audio.js          🌐 ITU-T Opus/G.711
|       |— dscp-config.js          🌐 RFC 6679 QoS
|       |— ice-config.js           🌐 RFC 5245
|       |— socket-config.js        🌐 IEEE 802.1p
|       |— performance-targets.js  🌐 ITU-T Y.1541
|       |— coturn-config.js        🌐 RFC 5389, 5766
|       |
|       |— services/
```

- | | | — mediasoup-wrapper.js  RFC 3550, 3551
- | | | — srtp-handler.js  RFC 3711 SRTP
- | | | — comfort-noise.js  RFC 3389 CN
- | | | — sdp-handler.js  RFC 4566 SDP
- | | | — turn-service.js  RFC 5766 TURN
- | | | — ice-coordinator.js  RFC 5245
- | | | — bandwidth-estimator.js  RFC 8083
- | | | — qoe-calculator.js  ITU-T P.800 MOS
- | | | — qoe-tracker.js  ITU-T E.800
- | |
- | | — monitoring/
 - | | — network-metrics.js  RFC 2889
 - | | — dscp-monitor.js  RFC 6679
 - | | — ice-monitor.js  RFC 5245
 - | | — stun-monitor.js  RFC 5389
 - | | — turn-monitor.js  RFC 5766
 - | | — performance-monitor.js  ITU-T Y.1541
 - | | — vlan-monitor.js  IEEE 802.1Q
- |
- | — infrastructure/
 - | — coturn/
 - | — coturn.conf  RFC 5389, 5766

— vlan-setup.sh	🌐 IEEE 802.1Q
— tc-qdisc.sh	🌐 RFC 6679, IEEE 802.1p
— nginx-qos.conf	🌐 RFC 6679, IEEE 802.1p
└— nginx.conf	🌐 HTTP/QUIC reverse proxy

NEXT STEPS: Implementation Roadmap

★ PHASE 1: SOFTWARE STANDARDS (Weeks 1-2)

Priority: Create these FIRST before coding starts

1. **IEEE 830 (SRS)** - 1 day
 - Define all functional requirements
 - Write user stories with acceptance criteria
 - Document Cameroon-specific constraints
 - → Teams can start design & development
2. **IEEE 1016 (SDS)** - 2 days
 - Finalize architecture diagrams
 - Define API contracts
 - Document module interfaces
 - → Backend team can start coding
3. **IEEE 1012 (V&V Plan)** - 1 day
 - Define test strategy
 - Establish test coverage targets (>80%)
 - → QA can start writing tests in parallel
4. **IEEE 829 (Test Documentation)** - Ongoing
 - Create test case templates
 - Document test procedures
 - → QA uses for test execution

5. **IEEE 1028 (Reviews & Audits)** - Ongoing

- Establish code review checklist
- Set up audit procedures
- → Tech leads implement before/during development

PHASE 2: NETWORK STANDARDS (Integration)

Priority: Implement during backend development (Weeks 2+)

1. **RFC 3550/3551 (RTP/Payload)** - Mediasoup integration
 2. **RFC 3711 (SRTP)** - DTLS key exchange
 3. **RFC 4566 (SDP)** - Session negotiation
 4. **RFC 5245/5389/5766 (ICE/STUN/TURN)** - NAT traversal
 5. **RFC 8083 (Congestion Control)** - Adaptive bitrate
 6. **ITU-T Standards** - Codec selection, QoE monitoring
 7. **IEEE 802.1p/Q** - QoS network configuration
-

Recommended Action RIGHT NOW:

Start with IEEE 830 (SRS) because:

-  Unblocks entire team (all need to know requirements)
-  Prevents scope creep (clear "done" definition)
-  Enables parallel development (backend + frontend can work simultaneously)
-  Provides test cases foundation (QA can write tests same day)

Then immediately: IEEE 1016 (SDS) - so backend can code to spec

Ready to create these documents?